

國立台北科技大學

微處理機實習期末專題

題目名稱：俄羅斯方塊

班級：電機三甲 組別：第 13 組

姓名：張博翔 b812111001

姓名：孫皓 110350450

日期：113.12.31

一.研究目的

我們的設計概念是設計一個可計分、計時的俄羅斯方塊，需使用 8*8 LED 陣列以及蘑菇頭，利用 8*8 陣列顯示遊戲內容，並以蘑菇頭控制方塊轉方向、加速向下與左右移動，搭配 LCD 液晶顯示模組進行目前分數顯示，並回傳分數，使用七段顯示器進行計時，在每次遊戲開始後，計錄所花的時間。利用蜂鳴器發出音效，讀取程式中變數顯示分數於 LCD，當執行完遊戲時，回傳資料後，使七節顯示器重新計時。也因此需要使用到 2 片 Arduino Uno R3 以及麵包板外接。一塊控制七段顯示器，另一塊控制蜂鳴器、LCD 液晶顯示模組、LED 陣列和蘑菇頭搖桿，最終目標是希望將遊戲與該功能進行連動，例如功能，才能完成俄羅斯方塊的挑戰。

二.軟硬體系統

軟體系統：

Arduino IDE，使用 Arduino 程式語言進行編寫

硬體系統：

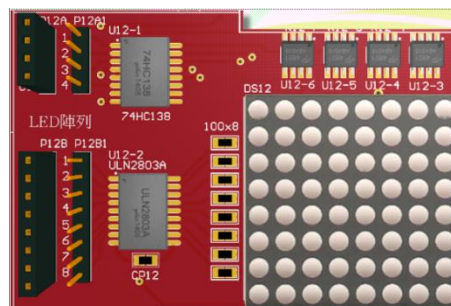
(1) Arduino Uno R3 (* 2)



(2) Arduino 16*2 I2C LCD



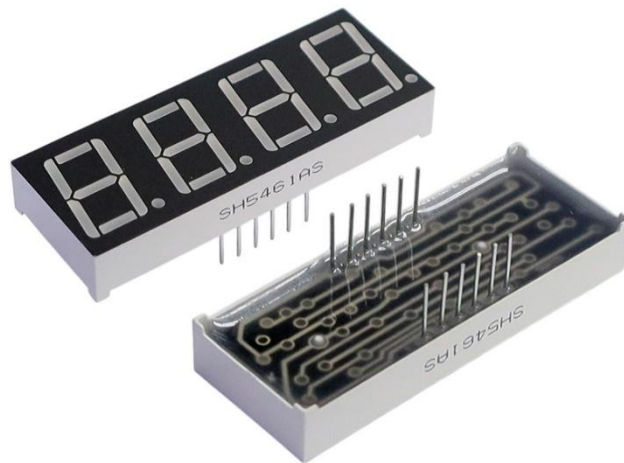
(3) 8*8 LED 陣列



(4) 蘑菇頭



(5) 七段顯示器

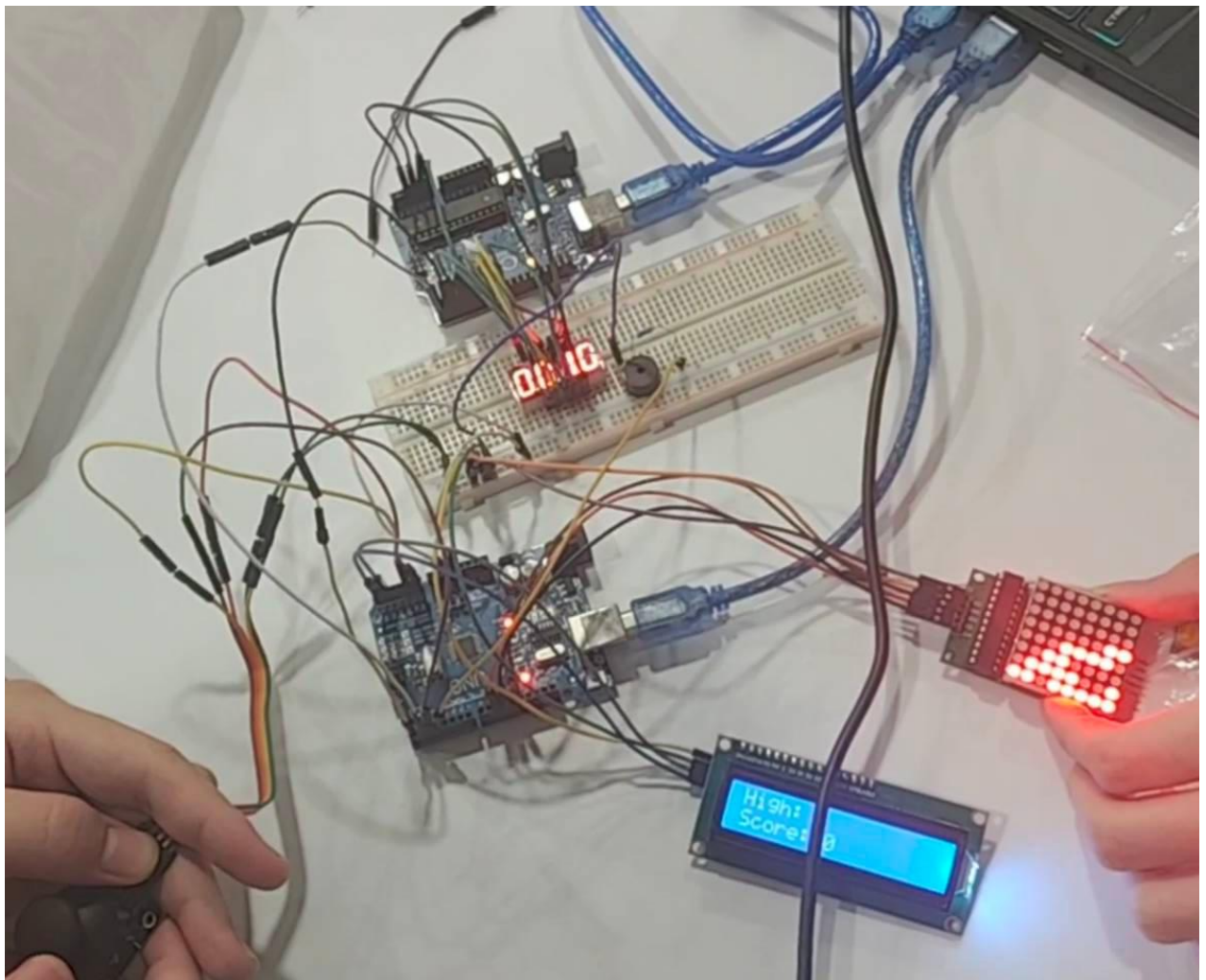


(6) 蜂鳴器



三.設計原理

接線圖



七段顯示器

負責記錄時間

- A-G 接到 Pin 2-8 (決定顯示什麼數字)
- D1-D4 接到 Pin A0 – A3 (決定哪一個數字會亮起)

8x8 點矩陣 (MAX7219)

用於顯示俄羅斯方塊遊戲畫面

- VCC: 接 5V
- GND: 接 GND
- DIN: 接 Pin 12
- CS (LOAD): 接 Pin 10
- CLK: 接 Pin 11

LCD 螢幕 (I2C 介面)

用於顯示分數、最高紀錄與遊戲狀態。

- VCC: 接 5V
- GND: 接 GND
- SDA: 接 A4
- SCL: 接 A5

磨菇頭搖桿 (Joystick)

用於控制方塊移動、旋轉與加速下落

- VCC: 接 5V
- GND: 接 GND
- VRx (X 軸): 接 A0 (控制左右移動)
- VRy (Y 軸): 接 A1 (控制旋轉與加速)
- SW (按鈕): 接 Pin 2 (用於開始遊戲)

蜂鳴器 (Buzzer)

用於播放移動、消除行與遊戲結束的音效。

- 正極 (+): 接 Pin 3
- 負極 (-): 接 GND

連接兩塊板子，完成兩塊板子之間的指令傳遞

- 通訊 RX Pin 0 (RX) 連接到 Board B 的 TX
- 通訊 TX Pin 1 (TX) 連接到 Board B 的 RX
- 共地 GND (GND兩塊 Arduino 的 GND 必須接在一起)

函式庫:

```
#include <SevSeg.h>
#include <LedControl.h>
#include <LiquidCrystal_I2C.h>
#include <Wire.h>
```

程式設計

(貼上程式文檔)

七段顯示器：

```
/*
 * Board B: Simple Timer (PC Control Only)
 * 功能：透過電腦 Serial Monitor 輸入 s 開始計時
 * 接線：
 * - 位數：A0, A1, A2, A3
 * - 筆畫：2, 3, 4, 5, 6, 7, 8
 */

#include <SevSeg.h>

SevSeg sevseg;

// ★★★ 極性設定 ★★★
// 如果顯示全亮或亂碼，請改為 COMMON_CATHODE（共陰）
#define DISPLAY_TYPE COMMON_ANODE

// 計時變數
unsigned long startTime = 0;
unsigned long elapsedTime = 0; // 經過的時間（毫秒）
bool isRunning = false;      // 是否正在跑

void setup() {
  Serial.begin(9600); // 啟動電腦通訊

  byte numDigits = 4;
  byte digitPins[] = {A0, A1, A2, A3};
  // 避開 Pin 12，使用 2-8
  byte segmentPins[] = {2, 3, 4, 5, 6, 7, 8};

  bool resistorsOnSegments = true;
  bool updateWithDelays = false;
  bool leadingZeros = true;
  bool disableDecPoint = true;

  sevseg.begin(DISPLAY_TYPE, numDigits, digitPins, segmentPins,
resistsOnSegments,
                updateWithDelays, leadingZeros, disableDecPoint);
  sevseg.setBrightness(90);

  Serial.println(F("System Ready."));
}
```

```

Serial.println(F("Type 's' to Start, 'p' to Pause, 'r' to Reset"));
}

void loop() {
  // 1. 顯示器刷新（最優先，無時無刻都要跑）
  sevseg.refreshDisplay();

  // 2. 檢查電腦有沒有輸入指令
  if (Serial.available() > 0) {
    char cmd = Serial.read();

    // 忽略換行符號
    if (cmd == '\n' || cmd == '\r') return;

    if (cmd == 's' || cmd == 'S') {
      // [Start] 開始計時
      if (!isRunning) {
        // 邏輯：當前時間 - 已經累積的時間 = 新的起跑點
        startTime = millis() - elapsedTime;
        isRunning = true;
        Serial.println(F("Timer Started!"));
      }
    }
    else if (cmd == 'p' || cmd == 'P') {
      // [Pause] 暫停
      isRunning = false;
      Serial.print(F("Paused at: "));
      Serial.println(elapsedTime / 1000);
    }
    else if (cmd == 'r' || cmd == 'R') {
      // [Reset] 歸零
      isRunning = false;
      elapsedTime = 0;
      sevseg.setNumber(0, 2);
      Serial.println(F("Timer Reset."));
    }
  }

  // 3. 計時邏輯
  if (isRunning) {
    elapsedTime = millis() - startTime;

    // 轉換成 秒數
    unsigned long totalSeconds = elapsedTime / 1000;

    // 顯示在七段顯示器上（純秒數 0-9999）
    sevseg.setNumber(totalSeconds);
  } else if (elapsedTime > 0) {
    // 暫停狀態，持續顯示最後的時間
    sevseg.setNumber(elapsedTime / 1000);
  }
}

```

```

} else {
    // 歸零狀態
    sevseg.setNumber(0);
}
}

```

俄羅斯方塊：

```

/*
 * Board A: Tetris Interface Master
 * 接線：
 * - LCD (I2C): SDA->A4, SCL->A5
 * - 搖桿: X->A0, Y->A1, Btn->2
 * - 矩陣: DIN->12, CLK->11, CS->10
 * - 蜂鳴器: Pin 3
 * - 通訊: 0(RX), 1(TX) -> 交叉接 Board B
 */

#include <LedControl.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// ★★★ 蜂鳴器 ★★★
#define BUZZER_PIN 3

// LCD 設定 (若沒畫面請改 0x3F)
LiquidCrystal_I2C lcd(0x27, 16, 2);

// MAX7219 設定
LedControl lc = LedControl(12, 11, 10, 1);

// 搖桿
const int joyX = A0;
const int joyY = A1;
const int joyBtn = 2;

// --- 紀錄變數 ---
unsigned long highScore = 0;           // 歷史最高分
unsigned long highScoreTime = 0;       // 創下最高分時的時間 (秒)
unsigned long currentScore = 0;        // 目前分數

// 遊戲變數
byte displayBuffer[8];
byte gameBoard[8];
int currentX = 2, currentY = -1, currentType = 0, currentRotation = 0;
unsigned long lastMoveTime = 0;

```

```

int dropSpeed = 500;

enum GameState { WAIT_START, PLAYING, GAME_OVER };
GameState currentState = WAIT_START;

// Tetris 形状
const byte shapes[7][4][4] = {
  { { B0000, B1111, B0000, B0000 }, { B0010, B0010, B0010, B0010 },
  { B0000, B1111, B0000, B0000 }, { B0010, B0010, B0010, B0010 } },
  { { B1000, B1110, B0000, B0000 }, { B0110, B0100, B0100, B0000 },
  { B0000, B1110, B0010, B0000 }, { B0100, B0100, B1100, B0000 } },
  { { B0010, B1110, B0000, B0000 }, { B0100, B0100, B0110, B0000 },
  { B0000, B1110, B1000, B0000 }, { B1100, B0100, B0100, B0000 } },
  { { B0110, B0110, B0000, B0000 }, { B0110, B0110, B0000, B0000 },
  { B0110, B0110, B0000, B0000 }, { B0110, B0110, B0000, B0000 } },
  { { B0110, B1100, B0000, B0000 }, { B0100, B0110, B0010, B0000 },
  { B0110, B1100, B0000, B0000 }, { B0100, B0110, B0010, B0000 } },
  { { B0100, B1110, B0000, B0000 }, { B0100, B0110, B0100, B0000 },
  { B0000, B1110, B0100, B0000 }, { B0100, B1100, B0100, B0000 } },
  { { B1100, B0110, B0000, B0000 }, { B0010, B0110, B0100, B0000 },
  { B1100, B0110, B0000, B0000 }, { B0010, B0110, B0100, B0000 } }
};

// 前向宣告
void showStandbyScreen();
void startGame();
void playTetrisLogic();
void triggerGameOver();
void refreshScreen();
void handleInput();
void updateDisplayBuffer();
boolean checkCollision(int x, int y, int rot);
void lockBlock();
void clearLines();
void spawnBlock();
void playTone(int freq, int duration);
void soundStart();
void soundClear();
void soundGameOver();
void soundMove();

void setup() {
  Serial.begin(9600);

  pinMode(joyBtn, INPUT_PULLUP);
  pinMode(BUZZER_PIN, OUTPUT);

  lc.shutdown(0, false);
  lc.setIntensity(0, 8);
  lc.clearDisplay(0);
}

```

```

    lcd.init();
    lcd.backlight();
    randomSeed(analogRead(A2));

    // 一開機先顯示待機畫面
    showStandbyScreen();
    Serial.println(F("System Ready. Waiting for 's' or Button."));
}

void loop() {
    // 1. 電腦輸入監控
    if (Serial.available()) {
        char c = Serial.read();
        // 輸入 's' 且在待機狀態 -> 開始
        if ((c == 's' || c == 'S') && currentState == WAIT_START) {
            startGame();
        }
    }

    // 2. 按鈕輸入監控
    if (currentState == WAIT_START && digitalRead(joyBtn) == LOW) {
        delay(50);
        if (digitalRead(joyBtn) == LOW) startGame();
    }

    // 3. 狀態機
    switch (currentState) {
        case WAIT_START:
            // 保持待機畫面，什麼都不做
            break;
        case PLAYING:
            playTetrisLogic();
            break;
        case GAME_OVER:
            // 結束畫面停留 3 秒
            if (millis() - lastMoveTime > 3000) {
                // 3 秒後自動回到待機介面
                showStandbyScreen();
            }
            break;
    }
}

// --- 介面與流程控制 ---

// 顯示待機畫面（歷史高分 & 時間）
void showStandbyScreen() {
    currentState = WAIT_START;
}

```

```

    lcd.clear();
    // 第一行：顯示歷史最高分
    lcd.print("HiScore:");
    lcd.print(highScore);

    // 第二行：顯示創下該紀錄的時間
    lcd.setCursor(0, 1);
    lcd.print("BestTime:");
    lcd.print(highScoreTime);
    lcd.print("s");

    // 傳送 Reset 給 Board B (歸零計時器)
    Serial.print('R');
}

// 開始遊戲
void startGame() {
    currentState = PLAYING;
    memset(gameBoard, 0, sizeof(gameBoard));
    currentScore = 0;
    spawnBlock();

    // 切換介面：顯示 最高分 & 目前分數 (時間隱藏)
    lcd.clear();
    lcd.print("High: "); lcd.print(highScore);

    lcd.setCursor(0, 1);
    lcd.print("Score: "); lcd.print(currentScore);

    // 傳送 Start 給 Board B (開始計時)
    Serial.print('S');
    soundStart();
}

// 遊戲結束
void triggerGameOver() {
    currentState = GAME_OVER;

    // 傳送 Pause 給 Board B，並請求時間
    Serial.print('P');

    // 讀取 B 板回傳的時間 (秒數)
    unsigned long gameDuration = 0;
    unsigned long waitStart = millis();
    while(millis() - waitStart < 1000) { // 等待最多 1 秒
        if(Serial.available()) {
            gameDuration = Serial.parseInt();
            while(Serial.available()) Serial.read(); // 清空 buffer
            break;
        }
    }
}

```

```

}

soundGameOver();

lcd.clear();
// 判斷是否破紀錄
if (currentScore > highScore) {
    // 破紀錄！更新分數與時間
    highScore = currentScore;
    highScoreTime = gameDuration; // ★ 更新最佳時間

    lcd.print("NEW HIGH SCORE!");
    lcd.setCursor(0, 1);
    lcd.print("Val: "); lcd.print(highScore);
} else {
    // 沒破紀錄
    lcd.print("GAME OVER");
    lcd.setCursor(0, 1);
    lcd.print("Score: "); lcd.print(currentScore);
}

lastMoveTime = millis(); // 用於計算停留時間
}

// 消除行數與更新分數
void clearLines() {
    int linesCleared = 0;
    for (int y = 0; y < 8; y++) {
        if (gameBoard[y] == 0xFF) {
            linesCleared++;
            for (int k = y; k > 0; k--) gameBoard[k] = gameBoard[k-1];
            gameBoard[0] = 0;
        }
    }
    if (linesCleared > 0) {
        currentScore += (linesCleared * 100);

        // 即時更新 LCD 分數
        lcd.setCursor(7, 1); // "Score: " 後面的位置
        lcd.print(currentScore);

        soundClear();
    }
}

// --- Tetris 邏輯 (不變) ---
void playTetrisLogic() {
    unsigned long currentMillis = millis();
    handleInput();
    if (currentMillis - lastMoveTime > dropSpeed) {

```

```

    if (!checkCollision(currentX, currentY + 1, currentRotation)) {
        currentY++;
    } else {
        lockBlock(); clearLines(); spawnBlock();
        if (checkCollision(currentX, currentY, currentRotation))
triggerGameOver();
    }
    lastMoveTime = currentMillis;
}
updateDisplayBuffer(); refreshScreen(); delay(20);
}

// --- 音效與硬體控制 ---
void playTone(int freq, int duration) { tone(BUZZER_PIN, freq, duration); }
void soundMove() { tone(BUZZER_PIN, 400, 10); }
void soundStart() { int melody[] = {262, 330, 392, 523}; for (int i = 0; i
< 4; i++) { tone(BUZZER_PIN, melody[i], 100); delay(120); } }
void soundClear() { tone(BUZZER_PIN, 1000, 50); delay(60); tone(BUZZER_PIN,
1500, 50); delay(60); tone(BUZZER_PIN, 2000, 100); }
void soundGameOver() { tone(BUZZER_PIN, 440, 200); delay(250);
tone(BUZZER_PIN, 349, 200); delay(250); tone(BUZZER_PIN, 293, 400); }

void refreshScreen() { for (int i = 0; i < 8; i++) lc.setRow(0, i,
displayBuffer[i]); }
void handleInput() {
    int xVal = analogRead(joyX); int yVal = analogRead(joyY);
    static unsigned long lastInputTime = 0;
    if (millis() - lastInputTime < 150) return;
    if (xVal < 100) { if (!checkCollision(currentX - 1, currentY,
currentRotation)) { currentX--; soundMove(); } lastInputTime = millis(); }
    if (xVal > 900) { if (!checkCollision(currentX + 1, currentY,
currentRotation)) { currentX++; soundMove(); } lastInputTime = millis(); }
    if (yVal < 100) { int nextRot = (currentRotation + 1) % 4; if
(!checkCollision(currentX, currentY, nextRot)) { currentRotation = nextRot;
playTone(600, 30); } lastInputTime = millis(); }
    if (yVal > 900) dropSpeed = 50; else dropSpeed = 500;
}
void updateDisplayBuffer() {
    for (int i = 0; i < 8; i++) displayBuffer[i] = gameBoard[i];
    for (int r = 0; r < 4; r++) {
        for (int c = 0; c < 4; c++) {
            byte rowShape = shapes[currentType][currentRotation][r];
            if (bitRead(rowShape, 3 - c)) {
                int worldX = currentX + c; int worldY = currentY + r;
                if (worldX >= 0 && worldX <= 7 && worldY >= 0 && worldY <= 7)
bitWrite(displayBuffer[worldY], worldX, 1);
            }
        }
    }
}
}
}
}

```

```

boolean checkCollision(int x, int y, int rot) {
    for (int r = 0; r < 4; r++) {
        for (int c = 0; c < 4; c++) {
            byte rowShape = shapes[currentType][rot][r];
            if (bitRead(rowShape, 3 - c)) {
                int worldX = x + c; int worldY = y + r;
                if (worldX < 0 || worldX > 7 || worldY > 7) return true;
                if (worldY >= 0) { if (bitRead(gameBoard[worldY], worldX)) return
true; }
            }
        }
    }
    return false;
}

void lockBlock() {
    for (int r = 0; r < 4; r++) {
        for (int c = 0; c < 4; c++) {
            byte rowShape = shapes[currentType][currentRotation][r];
            if (bitRead(rowShape, 3 - c)) {
                int worldX = currentX + c; int worldY = currentY + r;
                if (worldY >= 0 && worldY <= 7 && worldX >= 0 && worldX <= 7)
bitWrite(gameBoard[worldY], worldX, 1);
            }
        }
    }
}

void spawnBlock() { currentType = random(0, 7); currentRotation = 0;
currentX = 2; currentY = -1; dropSpeed = 500; }

```

四.實驗結果

我們的實驗成功透過磨菇頭搖桿精確控制俄羅斯方塊：往上撥動可改變方塊旋轉方向，往下撥動則加速下落速度，往左右撥動則控制方塊水平移動。這部分我們參考了課堂上老師教授過的電位計控制概念，將搖桿的類比數值轉換為邏輯指令，進而控制 8x8 LED 點矩陣上的座標偏移。

此外，針對俄羅斯方塊變換的多樣性，我們預先定義了七種經典方塊形狀的陣列，並配合 Arduino 的 random 隨機函數，結合未接線腳位 (A2) 的環境雜訊作為隨機種子，確保每次開機生成的方塊順序具有隨機性。

在整體系統架構上，我們採用了雙 Arduino 板通訊連動的設計。主控板 (Board A) 負責運算俄羅斯方塊的移動與消行邏輯，並透過 Serial 序列通訊 將啟動、暫停與重置指令傳送到顯示板 (Board B)。Board B 則專門處理計時邏輯，並驅動四位數七段顯示器。當我們完成消行獲得分數時，主控板會即時更新 I2C LCD 螢幕上的當前分數；而當遊戲結束 (Game Over) 時，主控板會讀取 Board B 回傳的總計時時間，並判斷是否刷新歷史最高分與最佳時間紀錄，成功實現了一個具備完整介面與音效反饋的互動遊戲系統。

五.結論

這次的實驗專題成功製作出一個能透過磨菇頭搖桿精確控制的俄羅斯方塊遊戲。在開發過程中，遇到了不少挑戰與困難。起初，在討論控制方式時，曾考慮過使用課堂教過的水平電位計或單純的按鈕，但為了追求更直覺的遊戲體驗，最後決定挑戰難度較高的搖桿控制。

在研發過程中，我們透過上網搜尋資料，克服了一些有關俄羅斯方塊的技術難題：

1. 方塊建模與旋轉：如何定義七種不同形狀的陣列，並確保方塊在旋轉時不會超出 LED 點矩陣的邊界。
2. 碰撞偵測與消行：設計邏輯判斷方塊何時接觸到地面，並在整行填滿時自動消除、更新分數。
3. 雙板系統聯動：透過序列通訊 (Serial Communication) 讓負責遊戲邏輯的 Board A 與負責計時顯示的 Board B 完美同步，實現遊戲與計時器的連動。

最後，在老師給予的這幾週時間內，充分利用所學，成功製作出心中理想的俄羅斯方塊系統。這學期的實驗課不僅讓我們掌握了 Arduino 的進階開發技巧，更讓我們學習到如何整合 LCD、點矩陣與七段顯示器等多種模組。從無到有完成這個專題，讓我們感到獲益良多，也達成了我們的實驗目標。

六.心得及分工

1. 學生 1 (負責報告編撰、影片剪輯與記錄)：我的主要工作是將複雜的研發過程轉化為專題報告，透過影片剪輯呈現遊戲的實際運作。參與調試過程，並記錄每一次調試的影片。在過程中了解各模組和電路的運作，
2. 學生 2 (負責硬體電路設計、多模組實作與偵錯)：我這次主要負責整體的硬體實作與電路開發。這是一個挑戰性極高的任務，因為我必須在同一個系統中整合 8x8 點矩陣、I2C LCD 螢幕、搖桿、蜂鳴器，以及另一塊 Arduino 上的四位數七段顯示器。最困難的部分在於處理雙 Arduino 之間的序列通訊 (Serial Communication) 與交叉接線，任何一個訊號干擾或地不穩都會導致計時與分數同步失敗。

	張博翔	孫皓
程式設計	O	
報告製作		O

七.參考資料

<https://webfish-channel.blogspot.com/2017/03/arduino3.html>

<https://projecthub.arduino.cc/candystan/8x8-tetris-e57938>

(7) **YouTube 連結**

<https://www.youtube.com/watch?v=N-O5XbSJ1nI>